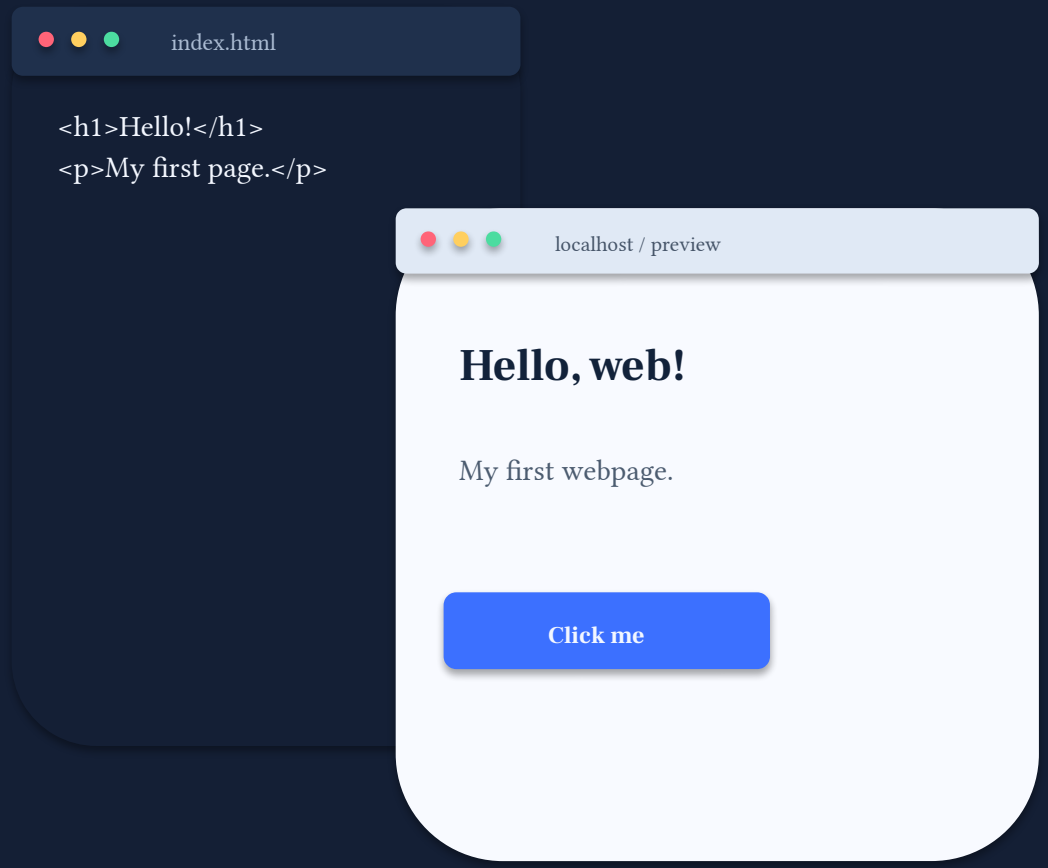
WEB3NOVA • HANDS-ON CLASS

WEB DEVELOPMENT FOR COMPLETE BEGINNERS

Understand what each line of code does — one step at a time.

READ → TYPE → RUN → UNDERSTAND



START HERE

Our learning method

Every new idea follows the same five steps.

READ

Look at the syntax

UNDERSTAND

Explain its job

TYPE

Write it yourself

RUN

Check the browser

CHANGE

Test one small edit

The goal is understanding, not typing quickly.

START HERE

The three building blocks

First learn what each language is responsible for.

HTML

Structure and content

CSS

Appearance and layout

JavaScript

Behaviour and interaction

HTML = what is there • CSS = how it looks • JavaScript = what it does

Set up your project

Keep all three files in the same folder.

project folder

```
web-beginner/
├── index.html
├── styles.css
└── script.js
```

1. Open VS Code.
2. Create a folder named web-beginner.
3. Create the three files shown.
4. Open index.html in a browser.
5. Keep editor and browser side by side.

Your first HTML: read every part

HTML tags tell the browser what kind of content to display.

index.html

```
<h1>Hello, web!</h1>
<p>I am learning.</p>
<button>Click me</button>
```

‹h1› … ‹/h1›

Creates a main heading. The words between the tags become visible text.

‹p› … ‹/p›

Creates a paragraph for a sentence or block of text.

‹button› … ‹/button›

Creates a button. It needs JavaScript to do something when clicked.

localhost / preview

Hello, web!

I am learning.

Click me

HTML syntax: opening tag, content, closing tag

Understand the pattern before memorising tag names.

one paragraph

```
<p>Welcome to class.</p>
```

Opening tag: `<p>`

Tells the browser a paragraph begins.

Content

The visible text: Welcome to class.

Closing tag: `</p>`

The slash / marks where the paragraph ends.

Most simple HTML elements follow this pattern.

A complete HTML document

The browser expects a basic structure around your content.

index.html

```
<!doctype html>
<html lang="en">
<head>
  <title>My First Page</title>
</head>
<body>
  <h1>Hello!</h1>
  <p>Welcome to my site.</p>
</body>
</html>
```

<!doctype html>

Tells the browser to use modern HTML.

<head>

Contains page information, such as its tab title.

<body>

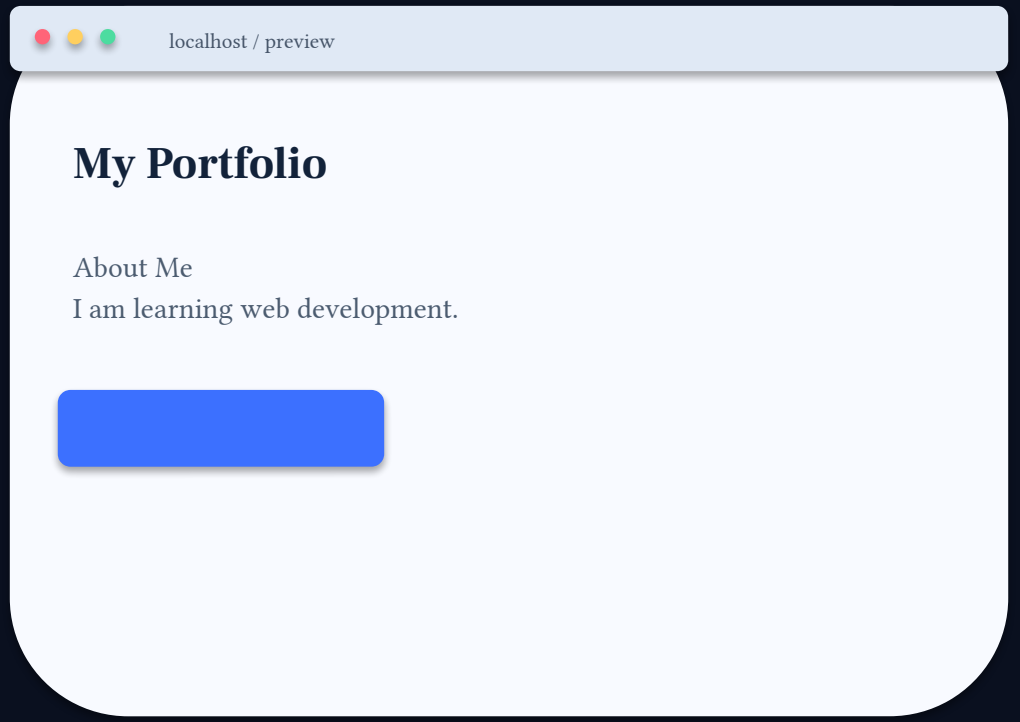
Contains the content shown on the page.

Headings and paragraphs

Use headings to organise sections and paragraphs for sentences.

index.html

```
<h1>My Portfolio</h1>
<h2>About Me</h2>
<p>I am learning web development.</p>
<h2>My Goal</h2>
<p>I want to build websites.</p>
```



h1 is the main heading; h2 is a section heading. CSS changes their appearance.

Lists: ul, ol and li

Use lists when you have related items or steps.

index.html

```
<h2>Tools I need</h2>
<ul>
  <li>VS Code</li>
  <li>A browser</li>
  <li>Practice time</li>
</ul>

<ol>
  <li>Write code</li>
  <li>Save</li>
</ol>
```

ul

Creates an unordered, usually bulleted list.

ol

Creates an ordered, usually numbered list.

li

Represents one item inside a list.

Links and the href attribute

The link text is clickable; href contains its destination.

index.html

```
<a href="https://example.com">Visit example</a>
<a href="about.html">About this website</a>
```

<a>

The anchor tag creates a link.

href='about.html'

The href attribute tells the browser where to navigate.

Visit example About this website

Images and attributes

An image uses src for its file and alt for a text description.

index.html

```
<img src="images/workshop.jpg"
     alt="Learners working on laptops"
     width="400">
```

src

The image file path or URL.

alt

A useful description for people who cannot see the image.

width

Sets the image width; responsive CSS is usually better for real layouts.

img is a void element: it does not use a closing tag.

PRACTICE

Exercise 1: create your profile page

Type this yourself, then change the words.

index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Profile</title>
</head>
<body>
  <h1>Your Name</h1>
  <p>I am learning web development.</p>
  <h2>My goal</h2>
  <p>I want to build my first website.</p>
</body>
</html>
```

TASK 1

Replace Your Name with your display name.

TASK 2

Change the goal sentence.

TASK 3

Save the file and refresh the browser.

CSS: the language of appearance

CSS rules select HTML elements and apply visual styles.

styles.css

```
h1 {
  color: blue;
  font-size: 36px;
}

p {
  color: #334 155;
}
```

h1 and p

Selectors: choose which elements receive a style.

color: blue;

Property is color; value is blue. The semicolon ends the declaration.

{ ... }

Curly braces contain the declarations for the selector.

CSS syntax: selector, property, value

Read one rule carefully before adding more.

styles.css

```
p {
  color: blue;
  font-size: 18px;
}
```

Selector

p means choose paragraph elements.

Property

color is what you want to change.

Value

blue is the chosen colour.

Semicolon

; marks the end of a declaration.

Try changing blue to red, then 18px to 24px.

Connect CSS to HTML

The link element tells HTML which stylesheet to load.

inside <head>

```
<head>
  <title>My Profile</title>
  <link rel="stylesheet" href="styles.css">
</head>
```

rel='stylesheet'

Describes the linked file as a stylesheet.

href='styles.css'

Names the CSS file. Its path must be correct.

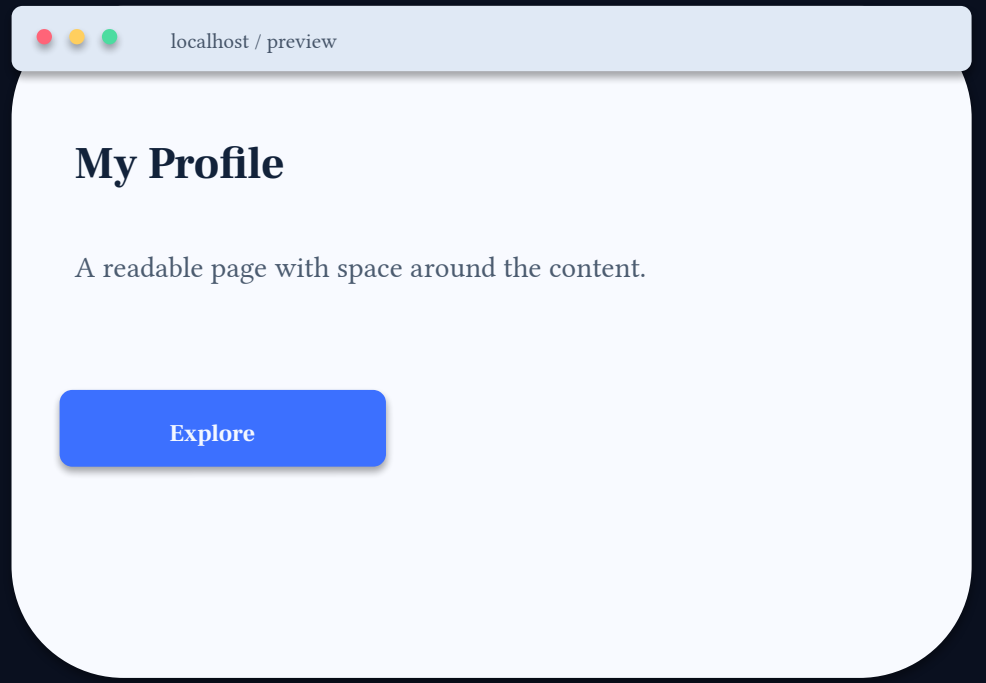
If styles do not appear, check the filename, spelling and folder.

CSS colour, font and spacing

These properties change how comfortable a page is to read.

styles.css

```
body {
  background-color: #f1f5f9;
  color: #172033;
  font-family: Arial, sans-serif;
  margin: 24px;
}
p {
  line-height: 1.6;
}
```



Classes: style a chosen group of elements

A class lets you apply the same style wherever it is used.

HTML + CSS

```
<!-- HTML -->
<p class="highlight">Important note</p>
<p>Normal paragraph</p>

/* CSS */
.highlight {
  background-color: yellow;
}
```

class='highlight'

Adds the class name to that paragraph.

.highlight

The dot tells CSS to select elements with that class.

HTML class: highlight | CSS selector: .highlight

The box model: where spacing comes from

Every element has content, padding, border and margin.



styles.css

```
.card {
  padding: 20px;
  border: 2px solid blue;
  margin: 16px;
}
```

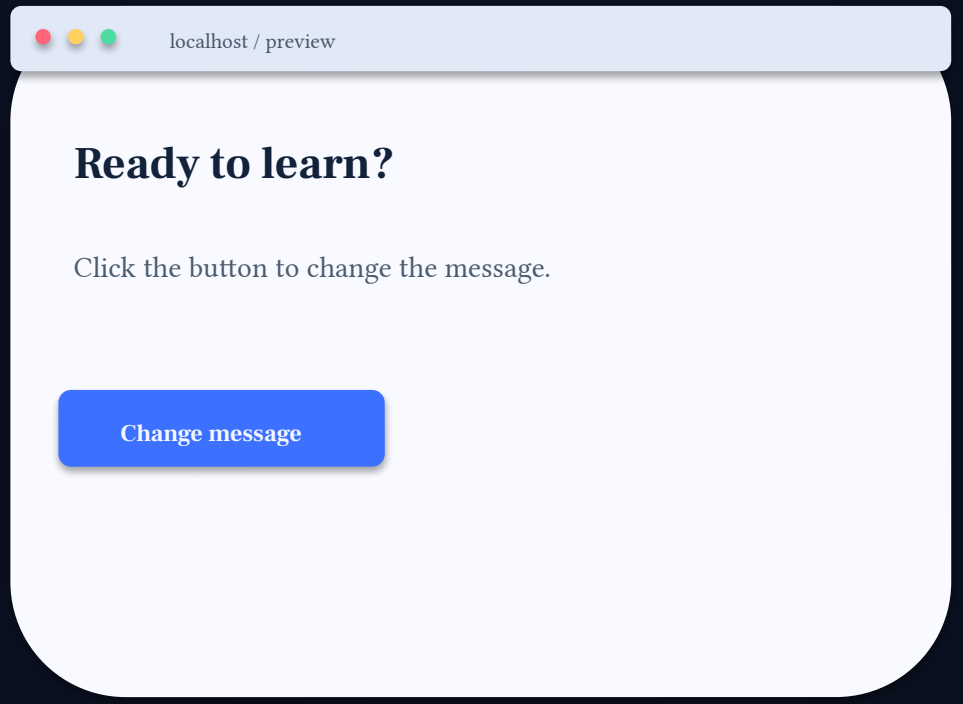
JavaScript: make a button respond

JavaScript can listen for a click and update the page.

HTML + JavaScript

```
<p id="message">Ready to learn?</p>
<button id="change">Change message</button>


const message = document.querySelector("#message");
const button = document.querySelector("#change");
button.addEventListener("click", () => {
  message.textContent = "You did it!";
});
```



JavaScript syntax: const and let

Variables give values names so the program can use them.

script.js

```
const learnerName = "Amina";
let lessonsCompleted = 1;
lessonsCompleted = 2;
console.log(learnerName);
console.log(lessonsCompleted);
```

const

Use when you will not reassign the variable.

let

Use when the variable may be reassigned.

console.log(...)

Prints a value in the browser console so you can inspect it.

JavaScript values: text, numbers, true/false

Different values represent different kinds of information.

script.js

```
const name = "Amina";   // string
const age = 20;         // number
const isLearning = true; // boolean
```

String

Text inside quotes.

Number

A numeric value such as 20.

Boolean

A value that is true or false.

JavaScript decisions: if and else

The program can choose which instructions to run.

script.js

```
const score = 8;
if (score >= 5) {
  console.log("You passed!");
} else {
  console.log("Keep practising.");
}
```

if (score >= 5)

Checks whether score is at least 5.

{ ... }

Groups the instructions for a condition.

else

Runs when the condition is false.

Functions: name a reusable action

A function groups instructions so you can use them again.

script.js

```
function greet(name) {
  return "Hello, " + name + "!";
}

const message = greet("Amina");
console.log(message);
```

function greet(name)

Defines a function named greet with an input called name.

return

Sends a result back to the code that called the function.

greet('Amina')

Calls the function and gives it the value Amina.

Debugging: what should you check?

Errors are clues. Read them and check one thing at a time.

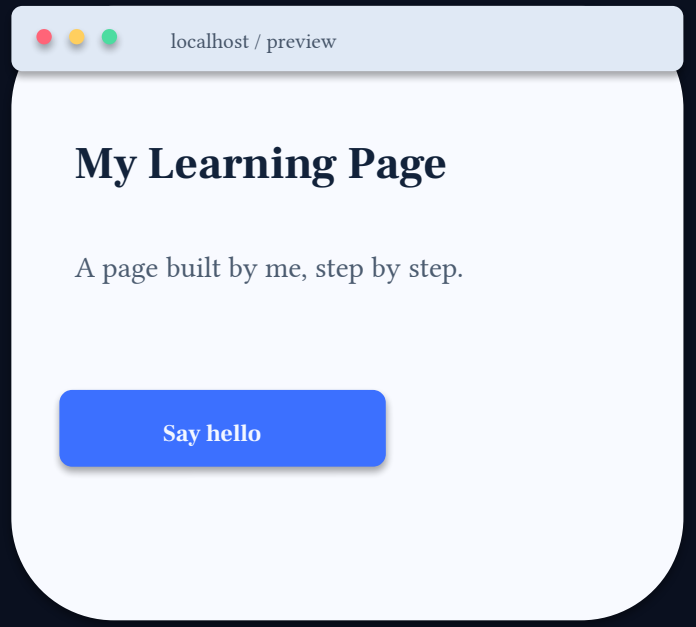
Nothing appears	Check HTML tags, then save and refresh.
CSS has no effect	Check the stylesheet link and filename.
Button does nothing	Check the element id and selector.
Image is missing	Check src and the image path.
Console shows an error	Read the message and inspect the named line.

YOUR TURN

Final practice: build your own profile page

Use the syntax you have learned. Make the page your own.

- Add a title and two headings.
- Write an introduction paragraph.
- Add a list of three learning goals.
- Add a link or image with useful alt text.
- Use CSS to change colour and spacing.
- Use JavaScript to change text after a click.



Quick review: explain each piece of syntax

Try explaining the purpose, not just naming the syntax.

`<h1> Title </h1>`	Creates a main heading.
`href='about.html'`	Specifies a link destination.
`p { color: blue; }`	Styles paragraph text blue.
`const name = 'Amina';`	Stores text in a variable.
`addEventListener('click', ...)`	Runs code when an event happens.

NEXT STEPS

What to practise after class

Build confidence through small daily exercises.

- HTML headings and paragraphs
- Lists, links and images
- CSS selectors and colours
- Spacing and layout
- JavaScript variables
- Functions and click events
- Finish your profile page

Understand one line at a time. Build something small. Repeat.

Facilitator notes: keep it gradual

The material is designed for explanation, demonstration and practice.

HTML	Teach tags and attributes; let learners practise before moving on.
CSS	Teach selector → property → value; show browser changes.
JavaScript	Start with variables, then conditions, functions and clicks.
Practice	Pause often and let learners type the code themselves.
Pacing	Split into multiple sessions if the group needs more time.

Do not rush to frameworks. Get comfortable with the basics first.

CLOSING

You can now read the code, not just copy it

The browser shows the result; your job is to understand the steps.

HTML = structure • CSS = style • JavaScript = behaviour

WRITE → SAVE → REFRESH → EXPLAIN

Web3Nova • Build your skills one line at a time.